CS3210 Assignment 2

By Sahej Agarwal (A0284719) and Ho Chi Thanh (A0276229W)

Program Design

Calculations across different rays for the interactions are occurring in parallel at the same time. Functions for construction of BVH nodes and computation and precomputation of triangles are also done in parallel.

Algorithm Overview

The program implements Monte Carlo path tracing for the assignment's Lambertian surfaces. Each sample follows a path from the camera into the scene, accumulating emitted light at each intersection. A throughput vector records how much light remains after reflection from previous surfaces. At each reflection, throughput is multiplied component by component by the object's albedo, and the next direction is generated using cosine-weighted hemisphere sampling.

When a path misses the scene, the background light is multiplied by the current throughput and added to the result. At the maximum permitted bounce depth, the program includes the current surface's emission and stops generating reflected rays. The implementation uses a loop rather than recursive function calls.

The main geometric operation is finding the closest triangle intersected by a ray. The supplied CPU implementation searches through the scene's triangles. Our GPU implementation instead uses a bounding volume hierarchy, or BVH. Each node stores an axis-aligned bounding box enclosing a group of triangles. If a ray misses that box, every triangle inside the corresponding subtree can be skipped.

The BVH is stored as a complete binary tree in an array. Node i has children at positions 2i + 1 and 2i + 2, so child pointers are unnecessary. Each populated leaf contains at most two triangles. Leaf capacity is rounded up to a power of two, and unused leaves are marked as empty.

Traversal uses a per-thread stack. When both child boxes intersect the ray, the nearer child is visited first and the farther child is deferred. Finding a nearby triangle early reduces the distance within which later intersections can improve the result, allowing additional subtrees to be pruned.

Optimization Attempts:

Optimization 1: Precomputed Triangle

The original triangle intersection routine calculates the two triangle edges whenever it tests a ray against a triangle. It also calculates and normalises the surface normal when an intersection succeeds. These quantities depend only on the triangle's vertices, which remain unchanged throughout rendering.

We therefore introduced GPUTriangle to calculate and store this information once during GPU preprocessing. It contains the first vertex, the two edge vectors, the unit surface normal and the minimum and maximum corners of the triangle's axis-aligned bounding box.

The intersection routine reuses the stored edges instead of repeatedly subtracting vertex positions. On a successful intersection, it reuses the stored normal and reverses its direction when necessary so that it faces against the incoming ray. BVH construction also reuses the stored triangle bounds.

This optimisation moves invariant calculations out of the repeated intersection work. Its benefit should become more significant as more rays encounter the same geometry. The trade-off is additional memory: with the current field definitions, GPUTriangle stores 80 bytes of data compared with 36 bytes for the original three-vertex Triangle. Larger records increase storage and the amount of data moved during sorting, so fewer arithmetic operations do not automatically guarantee a lower overall runtime.

The available benchmark results measure the complete implementation. They do not isolate triangle precomputation, so a separate before-and-after experiment is required to quantify this optimization's contribution.

Optimization 2: Morton Sorting and BVH Acceleration

A bounding volume hierarchy (BVH) groups nearby triangles so that rays can skip entire groups when they miss their bounding boxes. Large or overlapping boxes reduce this benefit by increasing unnecessary traversal.

We assign each triangle a Morton code using the centre of its axis-aligned bounding box (AABB), normalised to the scene bounds. Each coordinate is quantised to 10 bits, then interleaved into a 30-bit code. This maps 3D positions to a 1D ordering that generally preserves spatial locality. Appending the original triangle ID produces a unique 64-bit sorting key. The ID is also retained in `GPUTriangle` to identify the last hit triangle and resolve equal-distance intersections consistently. Thrust's `sort_by_key` sorts the keys and corresponding triangle records together on the GPU.

Consecutive sorted triangles form leaves containing at most two triangles. Leaf AABBs are computed in parallel, then parent bounds are combined bottom-up. Nodes within each level are processed in parallel, with ordered kernel launches between levels and no atomic counters.

This reduces ray-triangle intersection tests when bounding boxes exclude geometry, but sorting and construction add preprocessing costs. The benefit depends on whether subsequent ray traversal saves enough work to offset these costs.

Karras and Aila (2013) examine this tradeoff between construction speed and ray tracing performance, improving an initial BVH through treelet restructuring. Our implementation instead uses a fixed complete binary tree over Morton-sorted triangles, without treelet optimisation, triangle splitting or surface area heuristic (SAH) leaf selection. The paper therefore provides motivation rather than an algorithm reproduced in full.

Current measurements do not isolate the BVH's speed-up; this requires a controlled comparison against direct triangle searches.

Data Structures and Implementation Overheads

The CPU preprocesses the objects' triangle meshes into a contiguous triangle vector with object indices and computes scene bounds. Albedo and emission are stored once per object to avoid duplication.

GPU global memory stores triangles, BVH nodes, material properties, cached primary camera ray intersections, sample light intensities, pixel sums and output pixels. Temporary triangle, object-index and Morton-key arrays are freed after use; remaining allocations are freed after copying the output to the CPU.

The kernels use no explicit shared memory. Each thread reads geometry from global memory and maintains its own traversal stack, random state, throughput and accumulated light intensity. Register usage and local-memory spilling require profiling.

The sample array can dominate memory usage, requiring (12WHs^2P) bytes, where (W) and (H) are the output dimensions, (s) is the Supersampling factor, and (P) is Samples per pixel, the number of samples taken for each subpixel.

BVH construction incurs kernel-launch overhead at each internal level, with little parallel work near the root. Morton ordering groups nearby triangles but does not guarantee minimal bounding-box overlap or optimal traversal.

Areas for improvement

The current version of the algorithm has not yet been optimized for memory operations, especially when it comes to sorting and changing the locations of different GPU triangles.

At the same time, a single-ray traversal across the nodes of the BVH has not yet been optimized, as the traversal still remains linear and is highly dependent on the intersections.

Benchmarking

We were unable to get H100s for our benchmarking and focused on benchmarking using and thus focus on benchmarking with A100-80GB

Performance Check						
Scene	**Ours (s)**	**Bench 0 (s)**	**Bench 1 (s)**	**Bench 2 (s)**	**Bench 3 (s)**	**Bench 4 (s)**
cornell_box_2	2.89372	214.519	73.8027	53.3555	49.1478	13.1772
ufo	8.94052	139.047	67.1102	34.1304	31.6543	15.4104
dragon	20.025	96.1792	94.1216	80.7042	73.0172	41.935
chess	28.2578	650.556	199.27	149.536	132.961	249.01
cornell_box	11.7334	30.3613	10.5596	8.88009	7.77594	44.1609

Correctness Check				
Scene	**Runtime (s)**	**MSE**	**Threshold**	**Result**
debug_0	1.70317	0.0859375	1	PASS
debug_1	1.65202	0.046875	1	PASS
debug_2	1.66831	0.04824320475	1	PASS
cornell_box_2	2.89372	1.376637777	4	PASS
ufo	8.94052	0.6666666667	2	PASS
dragon	20.025	4.091681481	9	PASS
chess	28.2578	0.8611802469	2	PASS
cornell_box	11.7334	5.411053975	9	PASS

References

[1] Karras, T., & Aila, T. (2013, July). Fast parallel construction of high-quality bounding volume hierarchies. In *Proceedings of the 5th High-Performance Graphics Conference* (pp. 89-99)

[2]. NVIDIA. (n.d.). *Thrust*. NVIDIA Developer. https://developer.nvidia.com/thrust

Appendix

CS3210-A2-G11

AI Declaration

ChatGPT was used as a language enhancement and grammar-checking tool for the report.Generative AI was also used for this assignment in order to verify testing and experimentation. The ideas and insights remain original and cited in our report.

Use Codex for prompt formatting using the following prompt.

“Currently, my code inside path_tracing.cu is very untidy and difficult to read. Could you please format it properly using good indentation and probably reorder the allocation and deallocations of memory inside the The last function, in order to be able to get a good read of the program and make it more readable to the reader.”

Was also used as a beautification and formatting tool for the Excel sheets in the appendix.